Chapter 4: Data Types, Operators, Variables and Delimiters

In this chapter we introduce some of the basic building blocks of the C# programming language. In particular, we are going to take a look at variables, data types, operators and delimiters.

A variable is a programmatic symbol used to represent an object of a specific data type. Variables are so named because their value may change over time, i.e. vary. Every variable assumes a single data type meaning it can store values of a specific kind. In C# data types come in two flavors, primitive data types and user-defined data types. Primitive data types include things like strings, integers, reals, and dates. User-defined data types are also known as classes. Classes are built up from primitive data types and/or more simple classes. Classes are the cornerstone of object-oriented programming and will be covered in detail in Chapters 7 and 8.

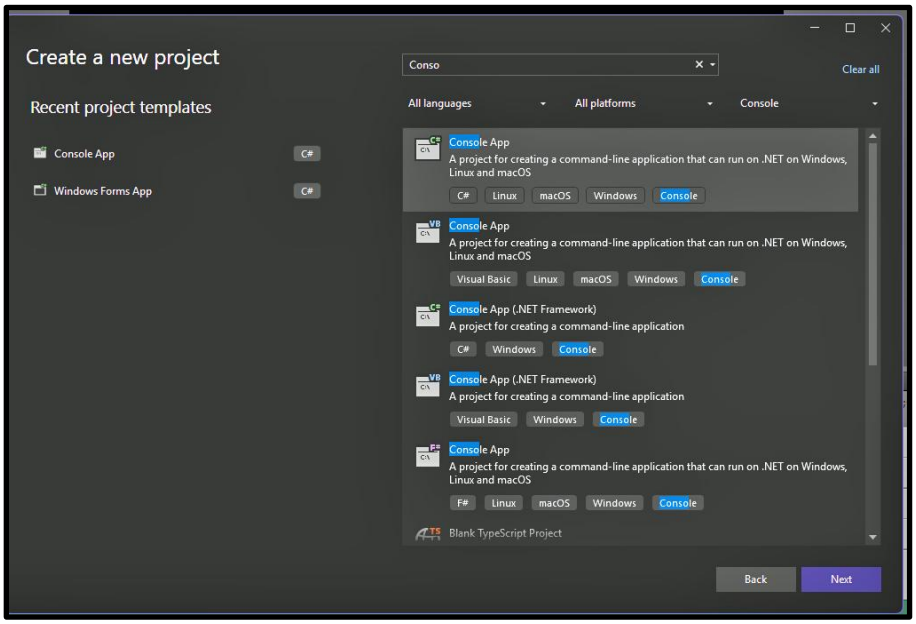
An operator is a symbol used to represent an action to be taken on one or more operands. Typical C# operators are adapted from mathematics, relational algebra and Boolean logic.

A delimiter is any symbol used to identify the beginning or end of a data type literal.

Okay, enough definitions, let's jump to the inside and look at some examples so this stuff can actually make some sense.

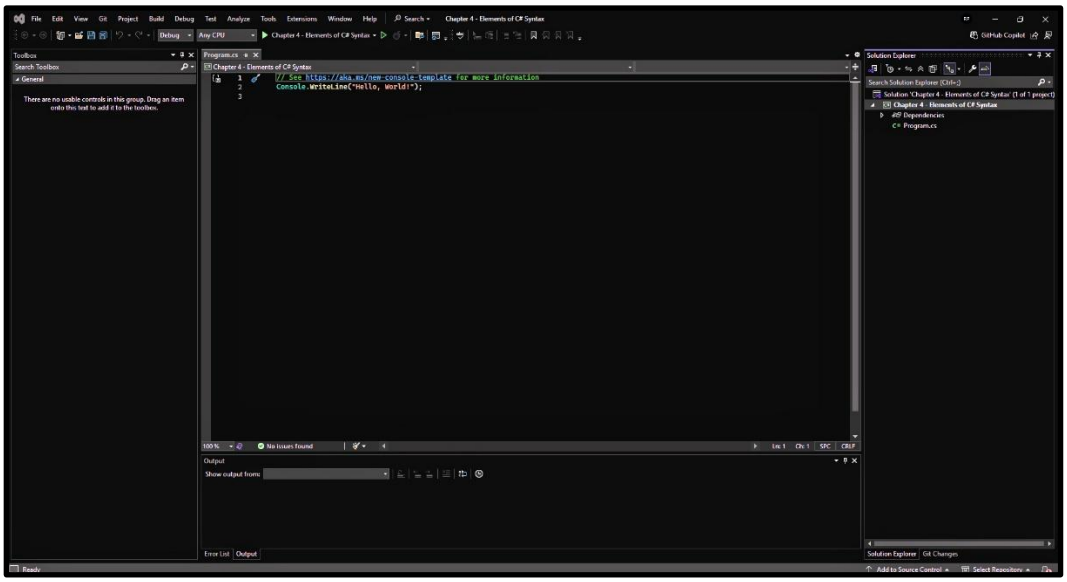
Please go ahead and open Visual Studio and choose to Create a New Project from the home screen. From the next page please choose to create a C# Console Application. By the way, did you notice that you can filter that long list of project types with that dropdown at the top? A lot easier than scrolling all the way through that endless list of project types. (Figure 4.0)

Figure 4.0: Console Application



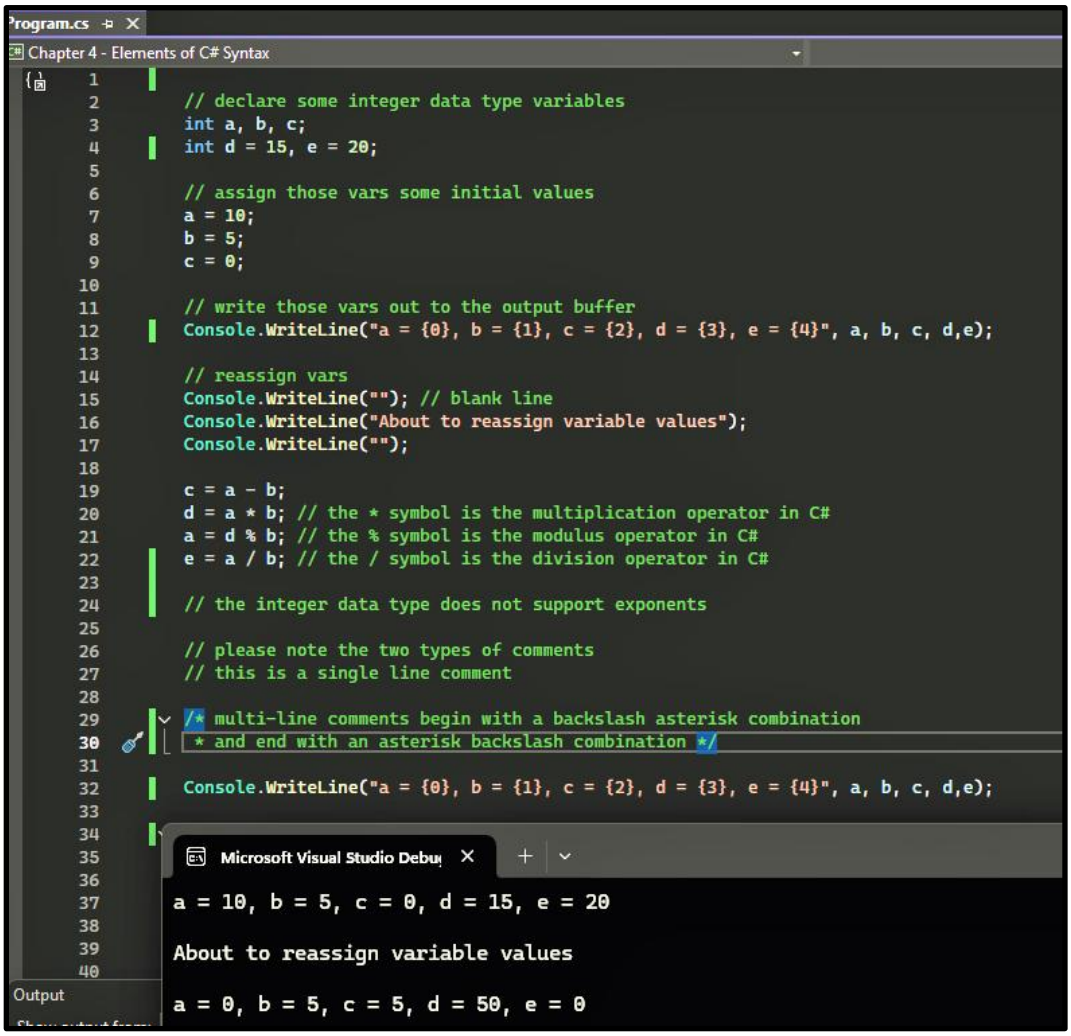
On the next screen let's call our project Chapter 4 – Elements of C# Syntax. Finally, select the default .Net framework and you should be looking at a screen similar to this. (Figure 4.1) What do we not see on this screen? A Form object. That is because C# console applications don't have a graphic component. They just serve as a container for code. Perfect, because that is exactly what we are going to write.

Figure 4.1: Console Application Environment



Please begin by deleting the two lines Visual Studio put there on your behalf, we don't need them. Next, please type in the code shown here wherein we introduce the integer data type. (Figure 4.2)

Figure 4.2: Integer Data Type



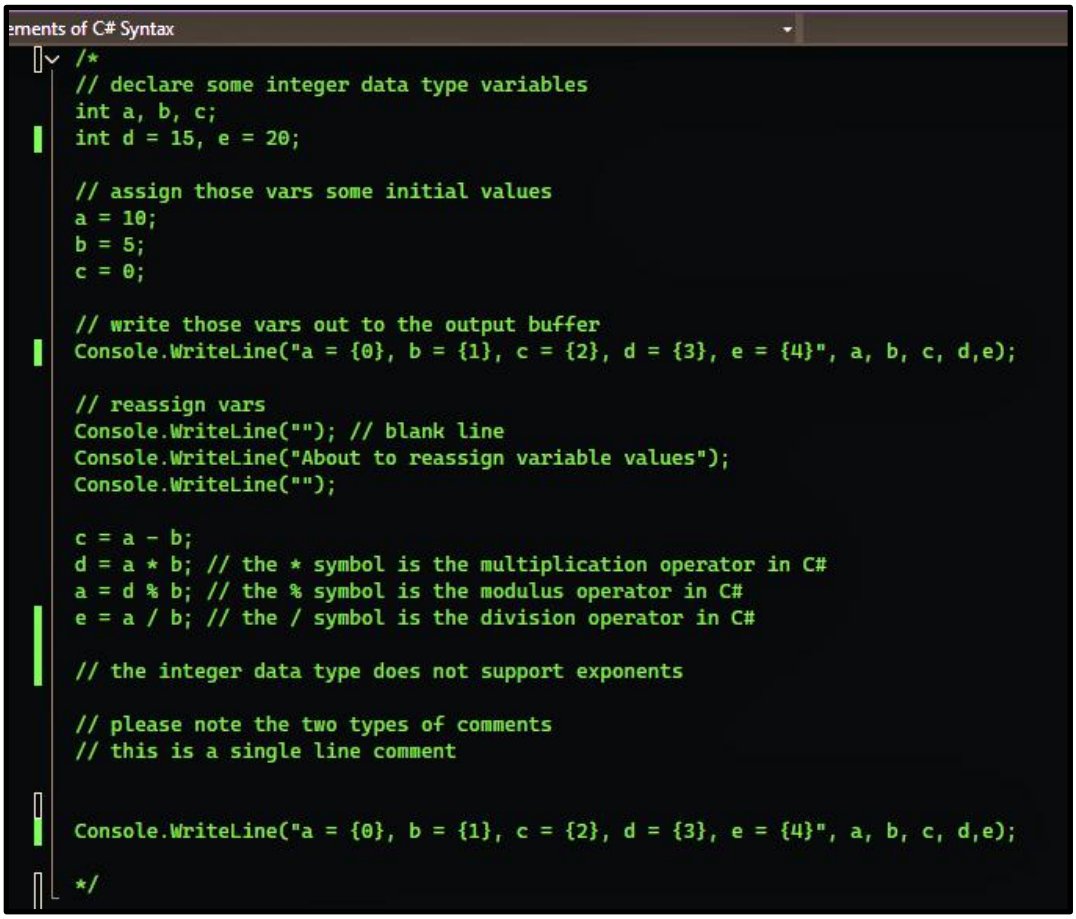
Okay, let's talk about what we can learn from this example.

- First, integer data type variables are declared in C# via the keyword int. Multiple variables may be declared in a single command. You may also declare and assign a value to a variable in a single command.
- Next, the Console.Writeline method accepts a string parameter and outputs that string to the output buffer.
- The a = {0} syntax specifies a variable replacement scenario where 0 represents the first variable to be replaced. C# is zero-based, so all numberings begin with the value 0.
- Next, the asterisk is C#'s multiplication operator, the backslash is C#'s division operator and the percentage symbol is C#'s modulus operator.
- There are two types of comments in C#. Single line comments start with two backslashes while multi-line comments start with a backslash asterisk combination and end with an asterisk backslash combination. Multi-line comments may be of any length.
- Finally, and probably most importantly, variables may assume new values over time as long as their data type remains consistent.

In case you are not familiar with the modulus operator, think of it as the remainder of a division operation.

Before we proceed onto real numbers, let's comment out the section of code we just wrote so it doesn't interfere with our new code. To do this put a backslash asterisk combination before the first line of code and an asterisk backslash combination after the last line of code. Multi-line comments cannot be nested so you will also have to remove the embedded multi-line comment. By the way, once you start getting serious about debugging, commenting out sections of code which you know are working properly can become a very helpful technique. (Figure 4.3)

Figure 4.3: Comment Out Integer Example

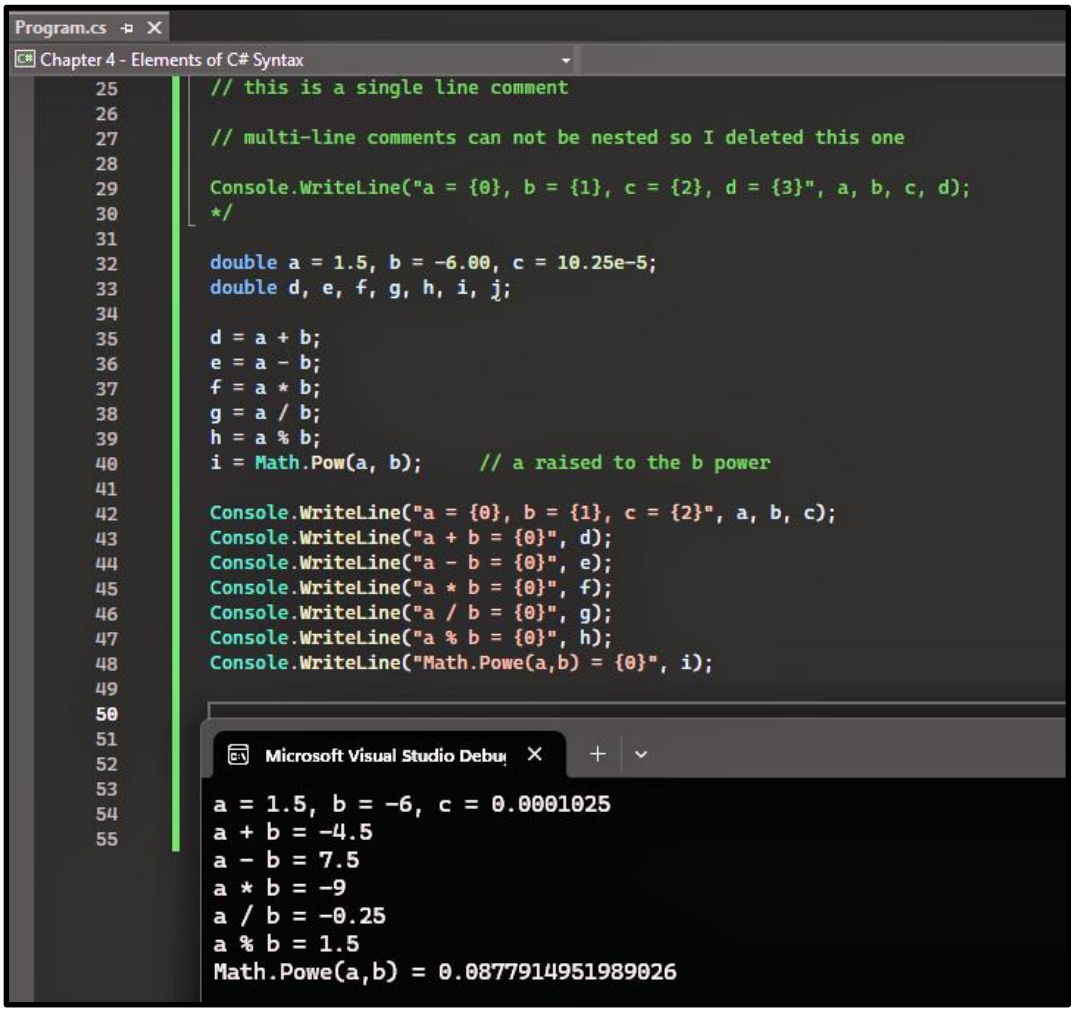


Our next example uses the real number data type which in C# is typically identified by the keyword double. Real numbers may also be identified by the keywords float or decimal but these are seldom used. I always used the double keyword. I have only seen the decimal keyword used in scientific applications where very high precision is required. (Figure 4.4)

Couple of interesting things going on in this example.

- First, in the top line we declare and instantiate three double variables in the same command. Pretty cool stuff.
- Next, unlike the int data type, the double data type supports exponents.
- Finally, the precision of a mathematical result in C# is not determined by the precision of the operands. To establish the precision of a mathematical result you must use the Math.Round() method.

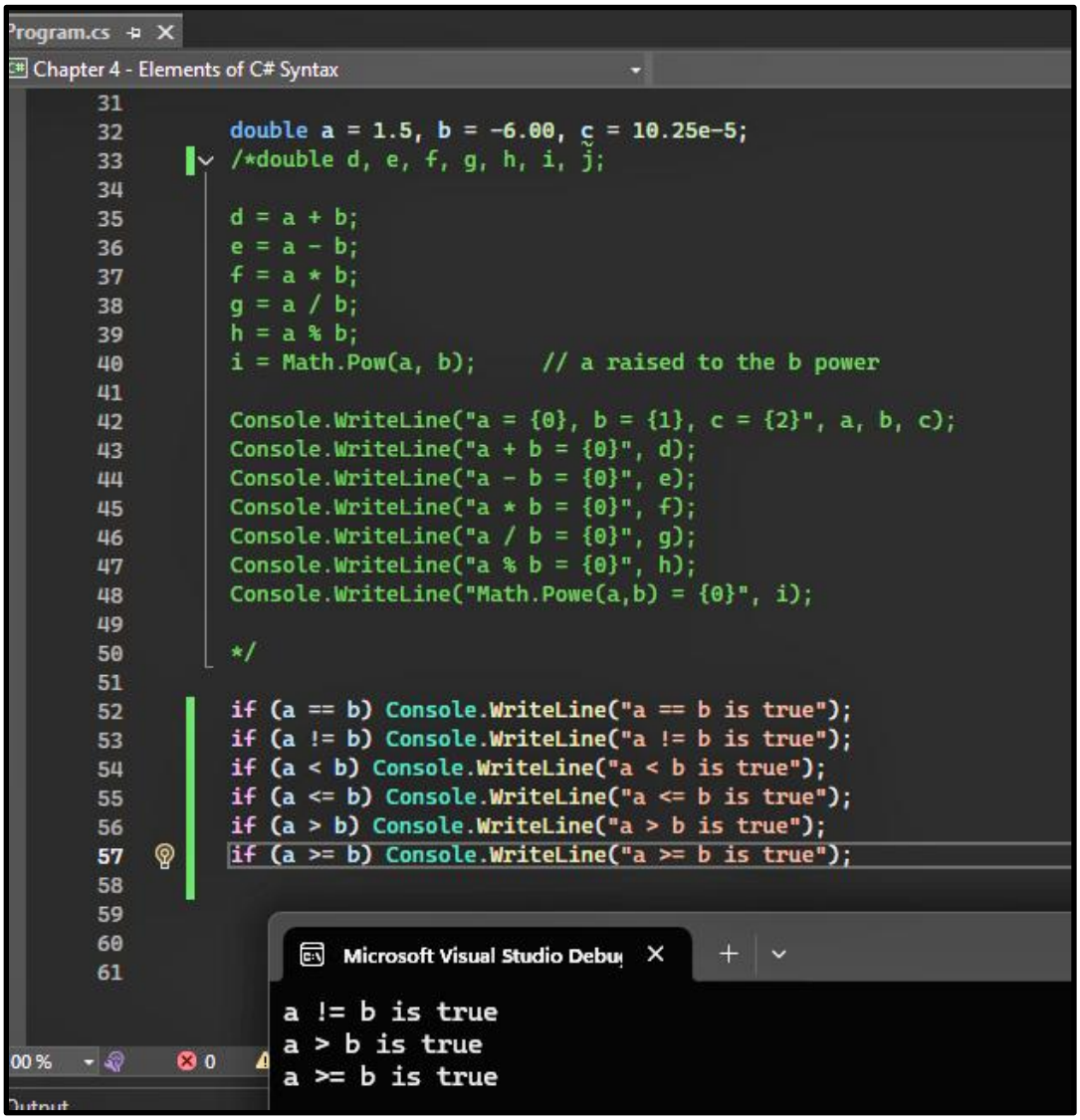
Figure 4.4: Double Data Type



In our next example we look at some relational operators. Before we proceed, go ahead and comment out everything in the last example except the first set of variable declarations and assignments. After that go ahead and add in the relational tests shown here. (Figure 4.5)

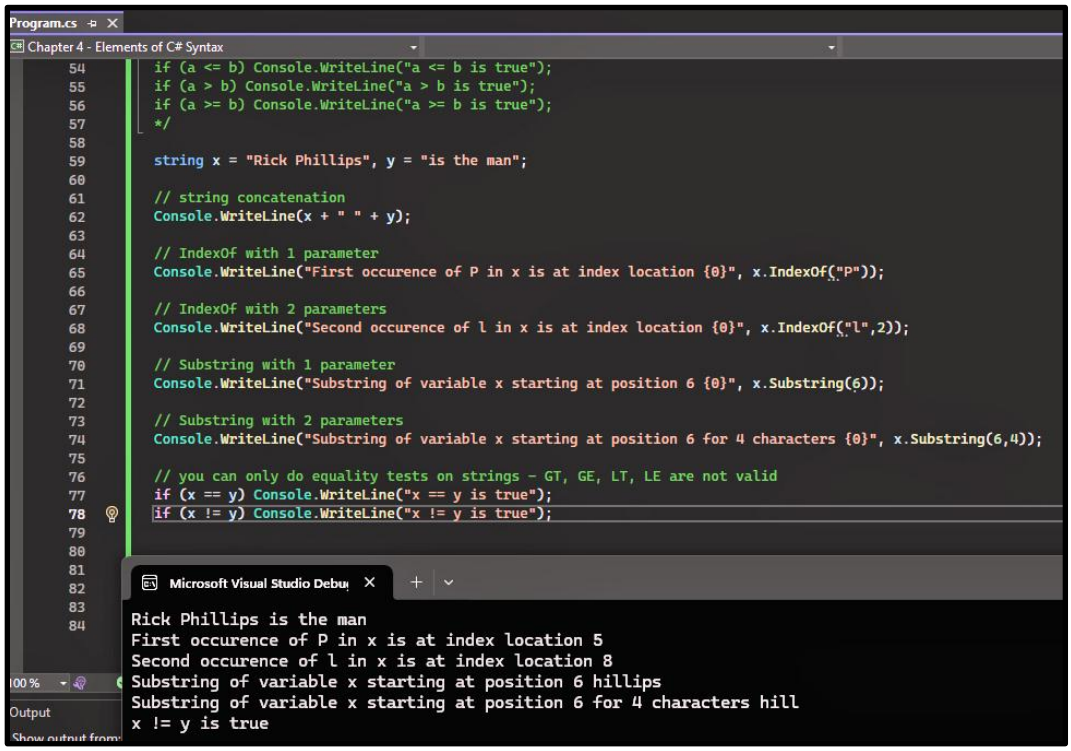
I really hope you are following along with your own code. Making those inevitable mistakes is part of the learning process. I know it can be frustrating, but it is worth it in the end. In addition, I hope you are anticipating the output prior to executing your code. Again, it is just really good practice and as our code becomes progressively difficult it is critical that you anticipate its behavior before testing.

Figure 4.5: Relational Operators



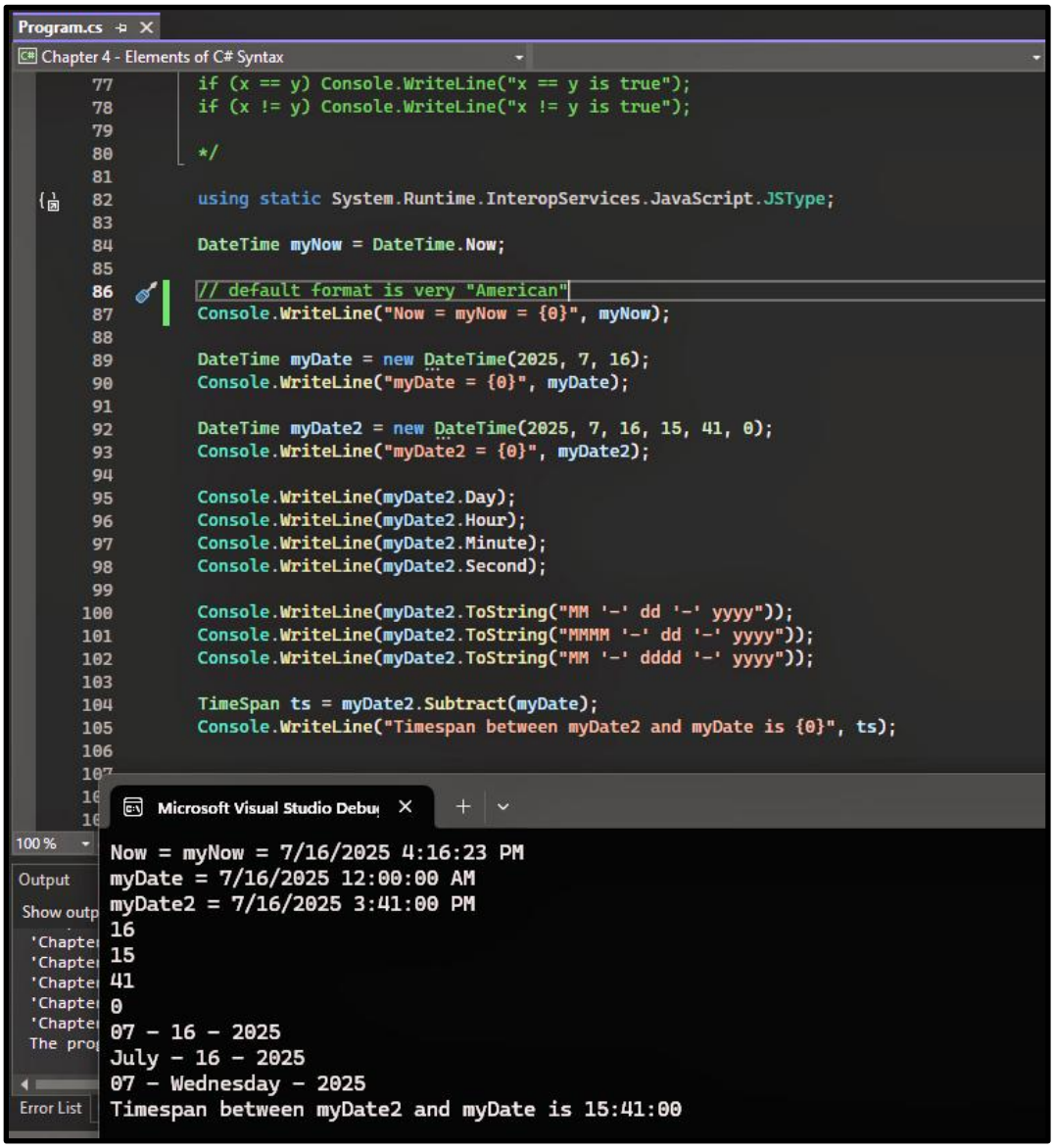
Okay, next we are going to move onto the string data type, but before we do go ahead and comment out everything we have done so far. Our string example is shown here. (Figure 4.6) Pretty straight forward stuff but syntax you will use nearly every day. By the way, did you notice my comment about equality tests and strings. Makes sense right, GT, GE, LT and LE really don't have any meaning when dealing with strings.

Figure 4.6: String Data Type



Okay, same game for this next example. Comment out everything we have done up to this point and go ahead and type in our DateTime data type example. (Figure 4.7)

Figure 4.7: DateTime Data Type



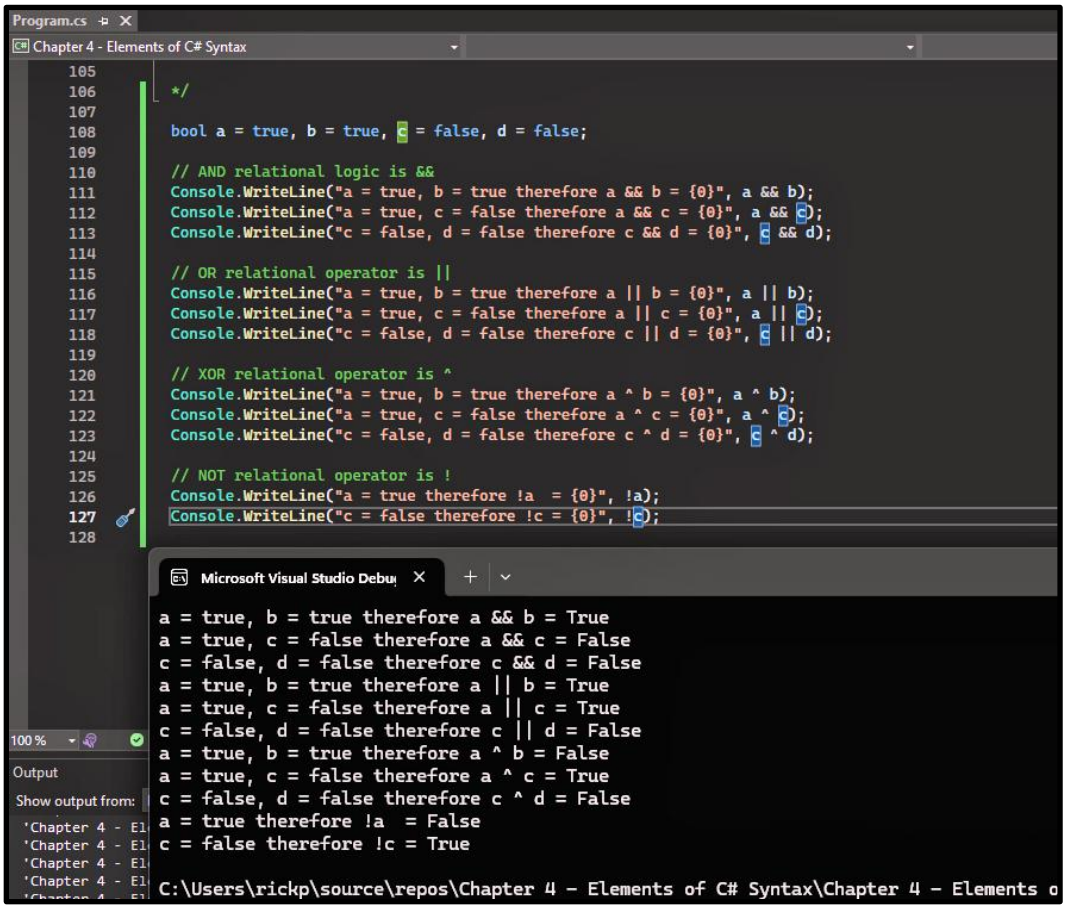
So, what did we learn from this example?

- First, that the default date format is very “Americanized”. Probably not too surprising given Microsoft is in Redmond, WA. Unfortunately, the format is not very practical for the rest of the world.
- Next, that you can format the output just about anyway you like. This is because dates are stored in C# as the number of intervals past or before a specific reference date. In fact, that date is January 1, 0001 at 00:00:00 Greenwich Mean Time. Intervals are measured in 100-nanosecond units. Microsoft refers to those 100-nanosecond units as ticks.
- The tick concept makes date and time arithmetic super easy because dates are actually nothing more than an integer.

We now move onto Boolean logic. There are two types of Boolean logic, two-state and three-state. In two-state logic a variable may be either true or false. In three-state logic a variable may be true, false or null. Three-state logic is typically used when dealing with relational databases which we will learn about later in this course. For now, we’ll first take a look at some two-state Boolean logic.

Here is our two-state Boolean logic example. Two-state Boolean logic is super easy. For an AND operator to be true both operands need to be true. For an OR operator to be true one or both of the operands must be true. For the XOR operator to be true one, but not both, of the operands must be true. Finally, the NOT operator simply reverses the value of its single operand. (Figure 4.8)

Figure 4.8: Two-State Boolean Logic

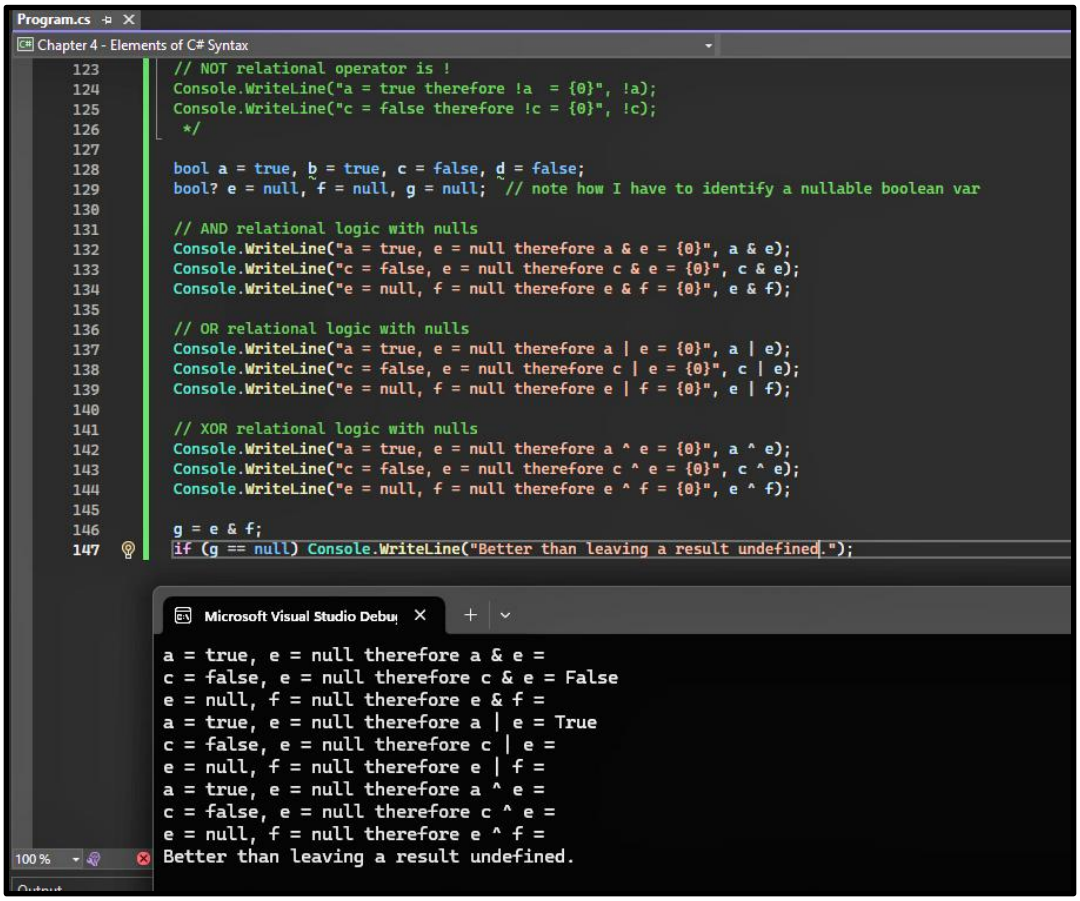


Let's conclude with a three-state Boolean logic example. As I just mentioned, we really won't deal with three-state logic until we get to relational databases, but rest assured that you need to know this stuff because relational databases will be a significant part of your professional life going forward.

In this example we see a lot of blank values. (Figure 4.9) That is because when C# can't determine the value of a logic test it returns nothing. Not sure who in the Microsoft world thought that was a good idea but trust me it is super dumb. Simply output the word undetermined or something like that, leaving it blank makes the user think they have made some type of error when in fact they have not. By the way, Microsoft does output a message when you have an indeterminant number operation, e.g. x/0. In this case you would get a NaN message, i.e. not a number.

So, what did we learn from this example? In short, that in most cases boolean logic with nulls returns a null or undetermined result. To really get an answer, test if the result is null and then deal with it appropriately. More on nulls when we start to return values from a relational database in Chapter 20.

Figure 4.9: Three-State Boolean Logic



All right, that concludes our variable, data type, operator and delimiter examples. Let's jump to the outside and do a quick review.

Okay, a lot of syntax to memorize right? Well, this stuff is all critical to your becoming a good C# programmer so strap on your big boy pants and get to work. This stuff simply must be memorized as it will be used on a daily basis when you start doing this professionally.

So, what did we learn?

- Variables assume a data type, either primitive or user-defined, and may vary over time.
- We looked at several primitive data types including integer (int), real (double), string, and DateTime.
- We also explored a bunch of differing operators including those associated with mathematics, relational algebra and Boolean logic.
- We did examples dealing with both two-state and three-state Boolean logic. A lot of the logic tests associated with null variables came back indeterminate so it is best to always test if a variable may be null.

All right, that concludes this lesson. When we meet next time we will be discussing program flow control, that is the order and frequency with which commands in a program are executed.